

Relazione tecnica

Titolo

Configurazione, contesto e flusso di analisi del progetto SO - Monitoraggio bancario in tempo reale.

Sommario esecutivo

Questa relazione descrive con taglio tecnico e operativo come **estrutturato il progetto**, da dove provengono i dati, in che modo vengono **normalizzati**, e come vengono trasformati in decisioni. Il documento non replica i dettagli dei singoli problemi (già presenti nel README), ma chiarisce il contesto, le motivazioni architetturali e il funzionamento complessivo.

L obiettivo edare una visione completa della pipeline: **ingresso dati**, **normalizzazione**, **monitoraggio**, **gestione del rischio** e **output finale**. L intero sistema è progettato per funzionare in tempo reale e con logica di escalation cumulativa, favorendo la tempestività rispetto all analisi storica.

1. Obiettivi del sistema

Gli obiettivi principali sono:

- monitorare un flusso di eventi in tempo reale;
- ridurre al minimo il tempo di rilevamento;
- definire regole semplici ma efficaci;
- accumulare il rischio nel tempo per evidenziare recidività;
- generare output strutturato e leggibile.

Il progetto simula un contesto bancario in cui le anomalie devono emergere in pochi secondi e non dopo una lunga elaborazione batch.

2. Contesto operativo

Il sistema assume un contesto in cui il server applicativo riceve richieste HTTP da client diversi (utente web, ATM, app, script automatizzati). Le richieste sono trattate come eventi con campi standardizzati.

In un ambiente reale, il monitoraggio puobasarsi su **traffico di rete**, **log applicativi** e **database**. Qui la scelta è stata deliberatamente semplificata: un solo canale in tempo reale, generato dal server, e letto in streaming dai monitor. Questa scelta rende l osservazione uniforme e consente di isolare la logica di rilevamento da fattori esterni.

3. Architettura logica

L architettura e' composta da tre blocchi principali:

1. **Server applicativo** (Flask)
2. **Flusso eventi** in `logs/realtime_access.log`
3. **Monitor** di sicurezza con output in `blacklist.csv`

La separazione enetta: il server produce eventi, i monitor leggono eventi, la blacklist raccoglie decisioni. Ogni blocco ha responsabilita singola.

4. Struttura del workspace

Percorsi principali:

- `server/server.py`: server Flask e generazione eventi.
- `logs/realtime_access.log`: stream eventi (input unico).
- `script/problema_*.sh`: monitor per ciascuna anomalia.
- `script/run_problem.sh`: orchestratore monitor + test.
- `script/test_generators/`: generatori di traffico per testing.
- `blacklist.csv`: output cumulativo.
- `clienti_banca.csv`: anagrafica (email e 2FA).

Il README descrive i problemi; questa relazione descrive il contesto e la configurazione globale.

5. Origine e formato dei dati

5.1 Generazione eventi

Il server riceve richieste HTTP (login, bonifico, prelievo, deposito). Ogni chiamata produce un evento e lo scrive nel file di stream.

Il server e' responsabile di:

- acquisire `customer_id` e parametri dell operazione;
- leggere l IP da `X-Forwarded-For` o `remote_addr`;
- generare un timestamp coerente;
- scrivere una riga nel log realtime.

5.2 Formato del flusso

Il formato scelto e' un record pipe-separated:

`timestamp|customer_id|ip|azione|importo|iban|session_duration`

Motivi della scelta:

- parsing semplice con strumenti standard (`awk`, `cut`);
- velocita' di lettura in streaming;
- compatibilita' con script bash.

5.3 Esempio concreto

2026-02-23T04:01:10|800001|192.168.10.12|LOGIN|||

Interpretazione:

- timestamp: 2026-02-23T04:01:10
- customer_id: 800001
- ip: 192.168.10.12
- azione: LOGIN
- importo: vuoto
- iban: vuoto
- session_duration: vuoto

6. Lettura dello stream

Ogni monitor usa:

```
tail -f logs/realtime_access.log
```

Questo comando mantiene una lettura continua del file mentre cresce. Non c'è buffering complesso: il monitor processa la linea appena appare.

6.1 Finestra temporale

Per evitare accumuli infiniti, ogni monitor applica una finestra di 60 secondi. Questo consente test rapidi e favorisce analisi a breve termine.

6.2 Stato temporaneo

Quando necessario, i monitor usano file di stato in `logs/` per contare occorrenze. Questi file sono puramente temporanei e non costituiscono input storico stabile.

7. Gestione del rischio

7.1 Blacklist

La blacklist è un CSV con header fisso:

```
timestamp,tipo_elemento,elemento,azione_rilevata,gravita,recidivita,risk_score,stato,origine
```

7.2 Recidivita' e rischio cumulativo

La logica centralizzata:

- legge l'ultima recidivita' per l'elemento;
- somma il nuovo rischio;
- aggiorna stato a BLOCKED se il totale è ≥ 100 .

Questo modello introduce una memoria operativa: un segnale medio ripetuto più volte può raggiungere criticità elevata.

8. Orchestrazione e test

8.1 run_problem.sh

Lo script di orchestrazione:

1. verifica server attivo;
2. avvia il monitor in background;
3. avvia il test generator;
4. termina il monitor dopo il primo alert.

8.2 Generatori di test

Ogni generatore:

- usa `curl` con parametri in query string;
- seleziona account casuali dal CSV;
- invia richieste in modo da superare la soglia.

I test sono pensati per essere riproducibili e immediati.

9. Motivazioni architetturali

9.1 Flusso unico

Un singolo flusso eventi semplifica la pipeline e garantisce coerenza. Invece di leggere da database o log multipli, si osserva un canale unico, che rappresenta l'azione in tempo reale.

9.2 Regole deterministiche

Le soglie sono statiche e chiare. Questo rende le decisioni spiegabili e favorisce il debugging e la validazione rapida.

9.3 Escalation cumulativa

L'accumulo di `risk_score` crea un ponte tra rilevamento immediato e memoria storica. Anche se il sistema non legge log storici, conserva la recidività nella blacklist.

10. Accenno ai problemi

I problemi sono documentati nel README. In sintesi, coprono:

- schemi AML
- accessi simultanei
- accessi notturni

- subnet ATM inattesa
- brute-force
- IP pubblici inattesi
- pattern API ripetitivi
- importo zero (covert)
- incoerenza ATM/bonifico
- low & slow

Questi casi sono stati scelti per coprire comportamenti, coerenza di rete ed attacchi a bassa intensita'.

11. Comandi principali e uso pratico

Comandi effettivamente usati nei monitor e test:

- `tail -f` per il flusso
- `awk -F` per parsing
- `sort -u` per unici
- `wc -l` per conteggio
- `grep -q` per match
- `cut -d -f` per estrazione
- `date -d` per interpretazione timestamp
- `date +%H` per orari
- `curl -s -G --data-urlencode` per generatori

Questi comandi garantiscono compatibilita' con un ambiente GNU/Linux standard e riducono dipendenze esterne.

12. Requisiti e dipendenze

Ambiente:

- Alpine Linux
- Bash
- Python 3
- Flask (server)
- Pandoc + LaTeX (solo per generare PDF)

Il monitoraggio non richiede librerie esterne oltre agli strumenti standard in shell.

13. Robustezza e limiti

Il progetto e' intenzionalmente semplice:

- non usa archiviazione storica come input;
- non calcola trend a lungo periodo;
- non integra feed esterni;
- non gestisce alert multipli simultanei.

Questi limiti sono accettabili nel contesto didattico e rendono la pipeline chiara e verificabile.

14. Conclusione

La configurazione proposta offre un contesto chiaro e pratico per studiare anomalie di sicurezza in tempo reale. I dati entrano dal server, vengono normalizzati nello stream, analizzati dai monitor e trasformati in output strutturati con accumulo del rischio.

La relazione mette in evidenza il flusso completo e il ragionamento architetturale, fornendo una base solida per eventuali estensioni future.

Allegato: esempio di riga blacklist

2026-02-23 04:03:46,ACCOUNT,800038,ACCESSO_NOTTURNO,MEDIA,1,30,ACTIVE,PROFILO_RETE,Accesso a

Fine documento.